

INTRODUCCIÓN A GIT Y GITHUB

Prof: Pereyra Pablo

E.E.S.T N° 3

PDISC

Introducción a Git

Introducción al control de versiones



El control de versiones es una herramienta que nos permite registrar cambios, organizar el trabajo, comprender la evolución de un proyecto y facilitar la colaboración entre múltiples personas. En proyectos profesionales es impensado trabajar sin un sistema como Git, ya que garantiza orden, trazabilidad y seguridad

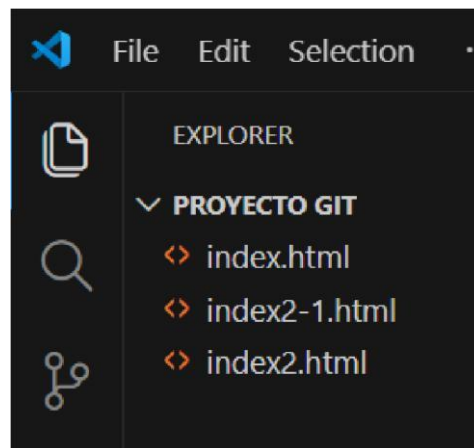
en todas las etapas del desarrollo.

¿Cómo funciona a nivel conceptual?

Podemos imaginar el control de versiones como una "línea del tiempo" del proyecto. Cada vez que guardamos un avance, estamos sacando una fotografía del estado actual de los archivos. Git almacena estas fotografías de manera eficiente, reconociendo qué cambió realmente y guardando sólo las diferencias necesarias.

¿Por qué es importante?

Sin control de versiones, es común terminar con archivos repetidos como por ejemplo:



Esto genera confusión, pérdida de información y dificulta la colaboración porque podríamos tener varias versiones de un mismo archivo repetidas, sin utilidad y sin sentido alguno.

Ventajas del control de versiones

- **Historial completo** de todos los cambios
- **Recuperación de versiones anteriores**

- **Trabajo colaborativo seguro**
- **Experimentación** sin riesgo de romper el proyecto principal

Git



¿Qué es Git?

Es el **sistema de control de versiones mas utilizado y reconocido**, que nos ayuda a guardar diferentes versiones de nuestro proyecto. Si tenemos algún error, podemos acceder a una de las versiones anteriores para volver a empezar. Esto se realiza a través del historial de versiones que se genera cada vez que se guarda una nueva versión.

Este flujo permite un control detallado. No estás obligado a guardar *TODO* de una vez, sino que podés preparar commits limpios y organizados.

Su creador Linus Torvalds, lo ideó con tres pilares fundamentales:

- **Velocidad:** Operaciones rápidas incluso con miles de archivos
- **Integridad:** Cada commit tiene un identificador único (hash) basado en su contenido
- **Flexibilidad:** Permite crear múltiples líneas de trabajo, fusionarlas y experimentar sin romper nada

Instalación y configuración inicial de Git

Instalación

Para poder utilizar esta herramienta en nuestro dispositivo, será necesario su instalación:

Si tenés Mac podés escribir en la terminal el comando `git` y seguir el paso a paso porque te ofrece una instalación automática. Si tenés Linux suele ver preinstalado y podés consultarlo ejecutando el comando `git --version` desde la terminal. En Windows será necesario realizar su instalación.

1. Para ello vamos a es necesario entrar al siguiente enlace: <https://git-scm.com/install/>.

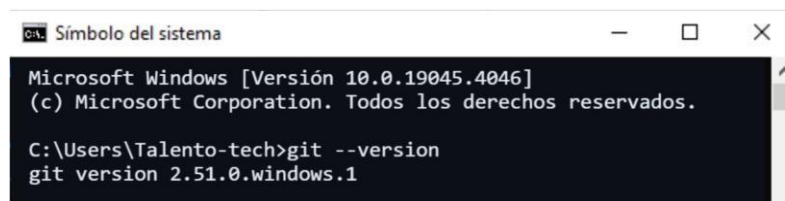
2. Elegir tu sistema operativo
3. Clickear sobre "Click here tu download". Esto reconocerá la arquitectura del sistema (32 bit o 64 bit) y te descargará un ejecutable.
4. Una vez descargado el ejecutable, seguís el paso a paso hasta confirmar la instalación.

Verificar instalación

Para poder verificar si tu computadora cuenta con Git instalado, solamente basta con escribir el siguiente comando en el CMD: **Command Prompt** o símbolo del sistema. También conocida como la terminal.

Comando: `git --version`

Ejemplo de respuesta: `git version 2.51.0.windows.1`



```
Microsoft Windows [Versión 10.0.19045.4046]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Talentotech>git --version
git version 2.51.0.windows.1
```

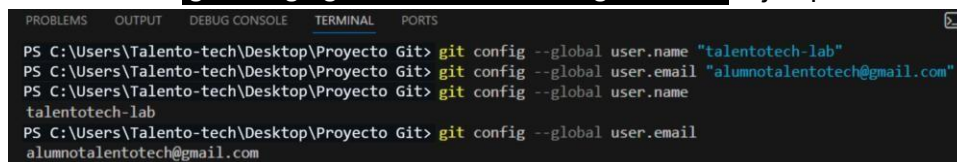
De lo contrario si no obtenemos la respuesta deseada y nos indica que no es conocido git, debemos volver a ejecutar el instalador o descargarlo.

Configurar identidad

La configuración de la identidad en Git es fundamental para identificar quién hizo los cambios en un repositorio, lo cual es crucial para la colaboración y el seguimiento del historial del proyecto. Al configurar un nombre de usuario y una dirección de correo electrónico, Git los registra automáticamente en cada "commit" que realizas, creando un registro de autoría claro para ti y otros colaboradores. Esto es vital en proyectos de equipo para saber quién contribuyó a cada cambio, y también es útil en proyectos personales para un seguimiento organizado.

`git config --global user.name "Tu Usuario"`

`git config --global user.email "tu@gmail.com"` Ejemplo:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Talentotech\Desktop\Proyecto Git> git config --global user.name "talentotech-lab"
PS C:\Users\Talentotech\Desktop\Proyecto Git> git config --global user.email "alumnotalentotech@gmail.com"
PS C:\Users\Talentotech\Desktop\Proyecto Git> git config --global user.name
talentotech-lab
PS C:\Users\Talentotech\Desktop\Proyecto Git> git config --global user.email
alumnotalentotech@gmail.com
```

Es recomendable usar el mismo correo y user que usaremos en GitHub.

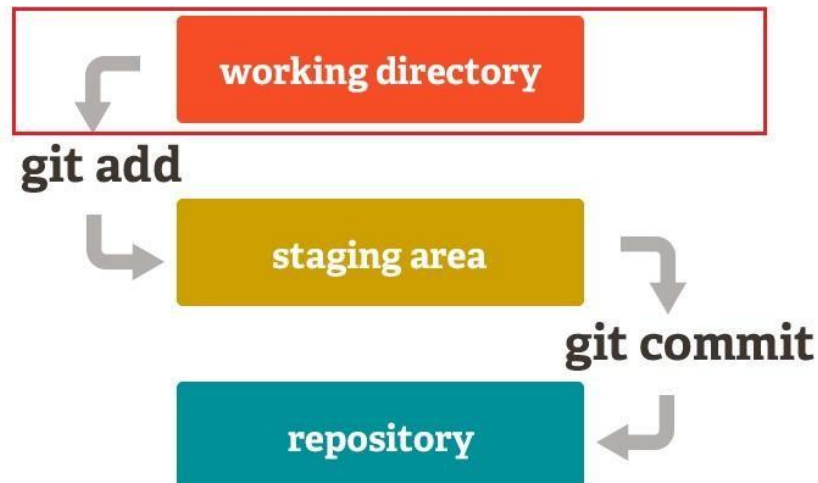
Verificar configuración

Para verificar que los datos se hayan ingresado correctamente y para contar con que efectivamente todos los cambios sean registrados correctamente ejecutamos: `git config --list`

Para comenzar a usar Git debemos entender su funcionamiento. Sabemos que es el encargado de controlar las versiones de las modificaciones que se van realizando dentro de un archivo o proyecto, pero ¿cómo funciona? Para eso, es necesario comprender los siguientes conceptos:

Conceptos claves sobre su funcionamiento:

- **Repositorio local:** Toda la historia y configuración del proyecto guardadas en tu PC de manera local.
- **Working directory:** El espacio donde trabajás y editas los archivos.
- **Staging área:** Una zona intermedia donde seleccionas qué cambios vas a guardar.
- **Repositorio (committed):** El lugar donde los commits quedan almacenados de forma permanente.



git add y **git commit** son palabras con las que nos empezaremos a familiarizar mientras avanzamos en el conocimiento de GIT.

Instalación de Visual Studio Code

Para comenzar a utilizar git, utilizaremos una interfaz que nos facilite la interacción entre los archivos que estemos trabajando y la terminal. Utilizaremos la terminal de Visual Studio Code, el editor de código mas utilizado actualmente. Para eso vamos a repasar su instalación:

Windows

1. Haz clic en "Download for Windows".
2. Ejecuta el archivo instalador (.exe).
3. Acepta el acuerdo y da clic en "Siguiente" hasta llegar a la pantalla de "Selección de tareas adicionales".
4. **Importante:** Marca todas las casillas (especialmente "Agregar 'Abrir con Code'..." y "Agregar a PATH").
5. Haz clic en Instalar y luego en finalizar.

macOS

6. Haz clic en "Download Mac Universal".
7. Abre el archivo .zip descargado (si no se descomprime solo).
8. Crucial: Arrastra el icono de Visual Studio Code a tu carpeta de Aplicaciones.
9. Ejecútalo desde "Aplicaciones" o Spotlight (Cmd + Espacio).

Tip extra: Ponerlo en español

Por defecto VS Code viene en inglés. Para cambiarlo:

1. Abre VS Code.
2. Presiona **F1** (o **Ctrl+Shift+P**).
3. Escribe "**Configure display language**".
4. Selecciona "**Install additional languages...**" y elige **Español**.
5. Reinicia el programa.

Iniciemos nuestro repositorio

Opción 1: Usando una consola (terminal)

- Abre una terminal (PowerShell, Git Bash o similar).
- Ubica en la carpeta donde quieres tu proyecto:

```
None  
cd nombre-de-la-carpeta
```

- Inicializa el repositorio:

```
None  
git init
```

- Verifica el estado del repositorio:

```
None  
git status
```

Esto crea un repositorio Git local en esa carpeta.

Registro de cambios: Flujo básico add + commit

Estados de los archivos

Estos estados describen en qué situación se encuentra cada archivo dentro del proyecto:

- **Untracked (Sin seguimiento):** Son archivos que existen en tu directorio de trabajo pero Git aún no los está controlando. Como archivos nuevos que todavía no los agregaste al repositorio; Git los detecta, pero no forman parte del historial de versiones.
- **Staged (Preparado):** Es el estado de los cambios que marcaste para incluir en tu próximo commit. Funciona como una etapa intermedia donde seleccionas exactamente qué modificaciones quieres guardar, permitiéndote revisar y elegir qué partes de tu trabajo pasarán al historial.
- **Committed (Confirmado):** Indica que los cambios que estaban en la zona de preparación ya fueron guardados en el historial del repositorio. En este estado, las modificaciones quedan registradas de manera permanente en la base de datos interna de Git.

Flujo básico

1. Crear o modificar un archivo (ej. `index.html`).
2. Ver estado: `git status`
3. Agregar cambios al área de preparación: `git add archivo` o `git add .`
4. Crear un commit: `git commit -m "Mensaje descriptivo"` Los mensajes deben ser claros y específicos.

Commit

Un commit es una acción para guardar los avances en el repositorio local, cada vez que se realice un commit se guarda la información junto al mensaje descriptivo, un identificador único (hash) y la referencia al commit anterior, formando así el historial del proyecto.

Historial de versiones y exploración

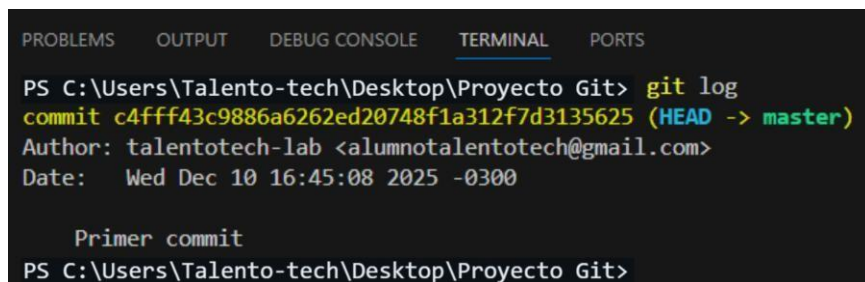
Ver historial simple

`git log`

Comando de Git que te permitirá ver el historial completo de los cambios (commits) que se realizaron, muestra una lista cronológica inversa, es decir del más reciente al más antiguo.

Cada commit muestra: Hash (recordemos era el código de identificación del commit), el autor indicando usuario e email, fecha incluido el horario y el mensaje que se agregó en el commit

Ejemplo:



```
PS C:\Users\Talento-tech\Desktop\Proyecto Git> git log
commit c4fff43c9886a6262ed20748f1a312f7d3135625 (HEAD -> master)
Author: talentotech-lab <alumnotalentotech@gmail.com>
Date:   Wed Dec 10 16:45:08 2025 -0300

    Primer commit

PS C:\Users\Talento-tech\Desktop\Proyecto Git>
```

Ver los cambios de un commit `git`

`show`

Muestra información detallada, también se puede ver el historial desde VS Code.

Ejemplo:

```

PS C:\Users\Talento-tech\Desktop\Proyecto Git> git show
commit d2dbcb3671d90e294a22afee88ceb691bb0a8942 (HEAD -> master)
Author: talentotech-lab <alumnotalentotech@gmail.com>
Date:   Wed Dec 10 16:59:24 2025 -0300

    Agregando título

diff --git a/index.html b/index.html
index d01f779..7fb289d 100644
--- a/index.html
+++ b/index.html
@@ -6,6 +6,7 @@
     <title>Document</title>
   </head>
   <body>
+    <h1>Titulo</h1>

   </body>
 </html>
\ No newline at end of file

```

- Cómo registrar cambios con commits.

Rol del repositorio remoto



Para comenzar, hablemos de qué es un repositorio remoto. En la clase 1 trabajamos únicamente de forma local, es decir, con repositorios guardados en nuestras propias computadoras. En esta clase empezaremos a trabajar de manera remota, lo que significa que nuestro repositorio tendrá una copia del proyecto alojada en un **servidor externo, que en este caso será GitHub**.

Un repositorio remoto nos sirve para:

- Respalda el proyecto.
- Compartir el código y colaborar.
- Hacer visible el avance ante otras personas.
- Sincronizar cambios desde distintos dispositivos.

¿Que es GitHub?

Es una **plataforma en la nube** que te permite guardar y gestionar proyectos, es donde vamos a guardar nuestro código, alojar repositorios remotos, colaborar y compartir tu código.

GitHub agrega:

- Hosting en la nube
- Herramientas visuales de colaboración
- Solicitudes de cambios (Pull Request),
- registro de tareas o problemas detectados,
- espacios para documentar el proyecto y automatizaciones

Pero el corazón del control de versiones sigue estando en Git mismo, por esta razón, durante este encuentro usaremos Git en local (nuestro equipo)

Comprendemos la relación entre:

- **Repositorio local** → Donde trabajamos.
- **Repositorio remoto** → Donde publicamos los cambios.

Creación de usuario en GitHub

Será necesario contar con una cuenta en Github si no la tenes aun podés crearla sin problema en el siguiente link:

Github: <https://github.com/login>

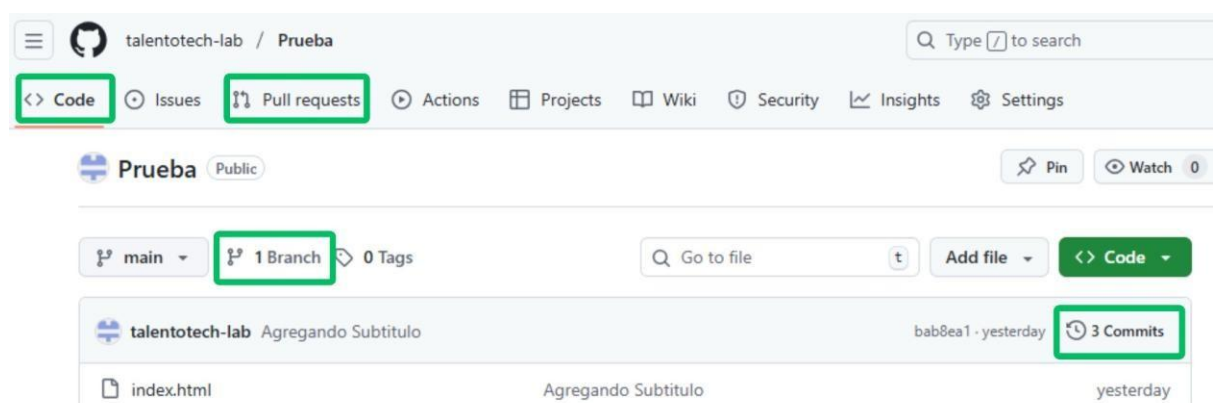
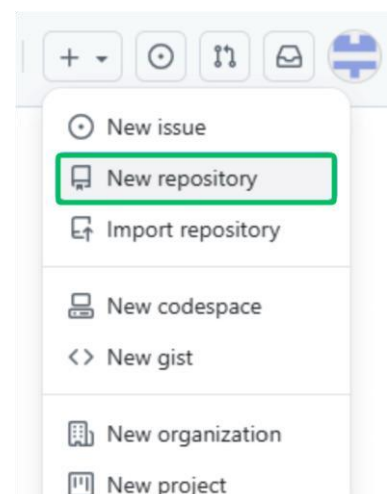
El email y el usuario que crees serán los que utilizaremos para poder comenzar a usar Git

Introducción práctica a GitHub

Ingresamos a la cuenta con la que estuvimos trabajando con Git y creamos un nuevo repositorio remoto.

Pasos iniciales:

1. Iniciamos sesión en GitHub.
2. Creamos un repositorio vacío para este proyecto.
3. Exploramos las secciones principales y que nos serán útiles del repositorio:
 - **Code**: archivos del proyecto.
 - **Commits**: historial de versiones.
 - **Branches**: ramas disponibles.
 - **Pull Requests**: solicitudes de fusión.



Para poder visualizar las secciones será necesario terminar de conectar con el repositorio.

Conectar el repositorio local con GitHub

Es necesario realizar la conexión para que nuestro trabajo que se encontraba de manera local pueda ser guardado en el repositorio remoto.

Comandos fundamentales:

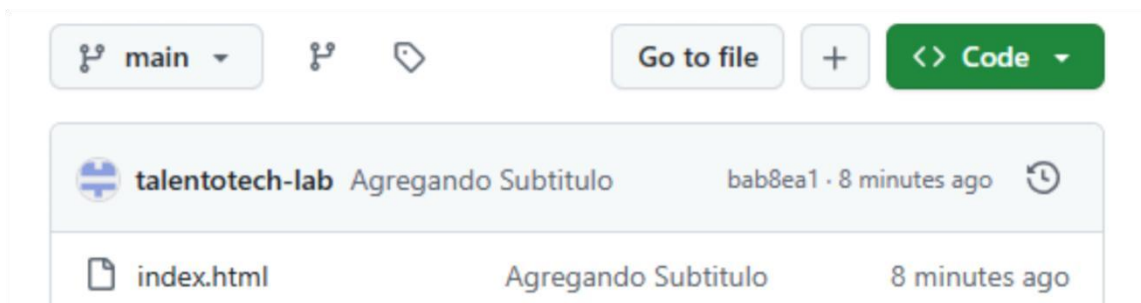
- Agregar el remoto: `git remote add origin <URL>`
- Branch: `git branch -M main` sirve para administrar ramas.
- Primer push: `git push -u origin main` la `-u`: establece a `origin/main` como predeterminado para futuros push y pull.

Ejemplo del código en la siguiente hoja:

```
git remote add origin https://github.com/talentotech-lab/Prueba.git
git branch -M main
git push -u origin main
```

Luego verificamos en GitHub que los archivos y los commits estén correctamente publicados.

Ejemplo de como se verá en github:



Una vez realizamos la conexión de nuestro repositorio local y nuestro repositorio remoto podremos ver todos los archivos, cambios y mensajes que fuimos dejando, que recordemos se los llamó commits.

Sign up for GitHub

Email

alumnotalentotech@gmail.com

[Use a different Google account](#)

Username*

talentotech-lab

Your Country/Region*

Argentina

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Email preferences

☐ Receive occasional product updates and announcements

Create account >

Ramas (branches) y trabajo paralelo

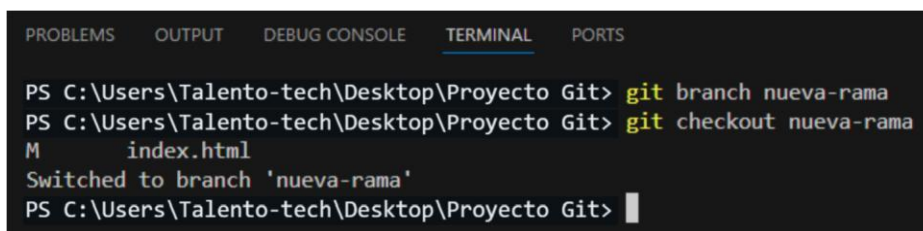
Una **rama** es una línea de trabajo separada que permite desarrollar nuevas ideas sin afectar la versión principal.

Cuándo usar ramas:

- Nuevas funcionalidades.
- Correcciones puntuales.
- Experimentos.

Creando una rama:

- Crear rama: `git branch nombre-rama`
- Cambiar de rama: `git checkout nombre-rama` o `git switch nombre-rama`



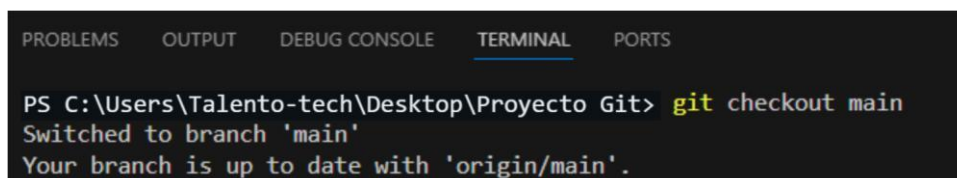
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Talent-tech\Desktop\Proyecto Git> git branch nueva-rama
PS C:\Users\Talent-tech\Desktop\Proyecto Git> git checkout nueva-rama
M      index.html
Switched to branch 'nueva-rama'
PS C:\Users\Talent-tech\Desktop\Proyecto Git> 
```

Con estos comandos creaste una rama nueva y cambiaste a esa rama, ahora puedes hacer modificaciones y solamente serán afectadas en esa rama y no en la rama principal **main**, puedes continuar trabajando y dejando commits de tus avances.

En caso de querer volver a la rama principal deberás escribir el siguiente comando: •

Volver a main: `git checkout main`



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Talent-tech\Desktop\Proyecto Git> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

Fusionar ramas (merge) y conflictos

El proceso de combinar los cambios de dos o más ramas en una sola es conocido como **MERGE**. Se utiliza comúnmente para integrar el trabajo de una rama que se creó para realizar alguna tarea en específico y combinarla en la rama principal (main o master) una vez que la funcionalidad está completa. Esto crea una única secuencia de commits que representa el historial de ambas ramas.

Paso a paso para fusionar:

1. Crear rama.
2. Trabajar y registrar cambios en esa rama.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Talentotech\Desktop\Proyecto Git> git status
On branch nueva-rama
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Talentotech\Desktop\Proyecto Git> git add .
PS C:\Users\Talentotech\Desktop\Proyecto Git> git commit -m "Agregando nueva rama"
[nueva-rama 5d5723e] Agregando nueva rama
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\Talentotech\Desktop\Proyecto Git>
```

3. Volver a `main`: Al momento de volver vamos a notar que todo el código o archivos en los que trabajamos desaparecen, pero realmente es que no se encuentran dentro de la rama principal conocida como main.
4. Ejecutar `git merge nombre-rama`: Esto lo hacemos para poder combinar la rama que nombremos con la rama principal, de esta manera lograremos ver todo lo que estuvimos trabajando.

```
PS C:\Users\Talentotech\Desktop\Proyecto Git> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Talentotech\Desktop\Proyecto Git> git merge nueva-rama
Updating bab8ea1..5d5723e
Fast-forward
 index.html | 4 ++++
 1 file changed, 4 insertions(+)
PS C:\Users\Talentotech\Desktop\Proyecto Git>
```

Conflictos

Ocurren cuando hay cambios superpuestos en el mismo archivo o línea de código: dos personas editan lo mismo. O cuando una rama edita un archivo que otra rama elimina; Git no sabe cuál versión mantener y requiere intervención manual para resolver la fusión.

- Error que aparece cuando se quiere cambiar de una rama pero no se realizó el commit correspondiente:

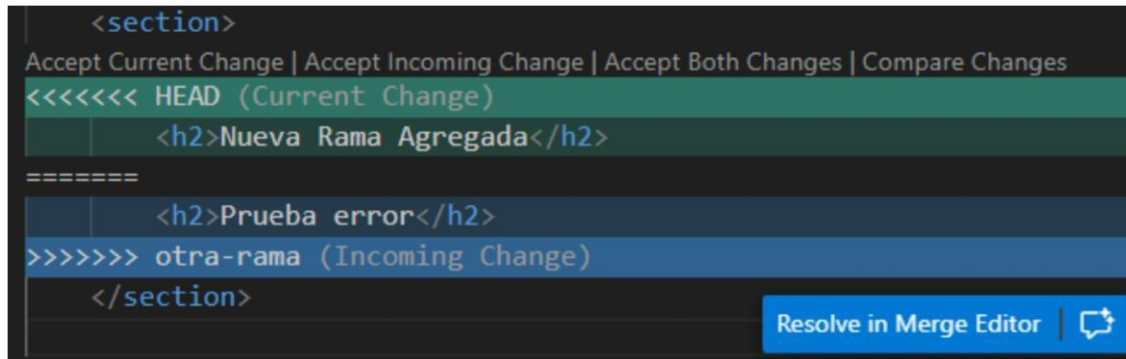
```
PS C:\Users\Talentotech\Desktop\Proyecto Git> git checkout otra-rama
error: Your local changes to the following files would be overwritten by checkout:
    index.html
Please commit your changes or stash them before you switch branches.
Aborting
PS C:\Users\Talentotech\Desktop\Proyecto Git>
```

- Error al momento de hacer una fusión (merge) cuando el código se sobreescribe:

Consola:

```
PS C:\Users\Talento-tech\Desktop\Proyecto Git> git merge otra-rama
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Talento-tech\Desktop\Proyecto Git>
```

Código en VSC:



La solución es realizar de manera manual, editando el código desde el editor, para que quede guardado pero no te olvides de escribir la secuencia de pasos de git add y git commit para guardar esa modificación

```
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Talento-tech\Desktop\Proyecto Git> git add .
PS C:\Users\Talento-tech\Desktop\Proyecto Git> git commit -m "Solucionando error"
[main 980abdd] Solucionando error
PS C:\Users\Talento-tech\Desktop\Proyecto Git>
```

Pull requests y revisión de cambios

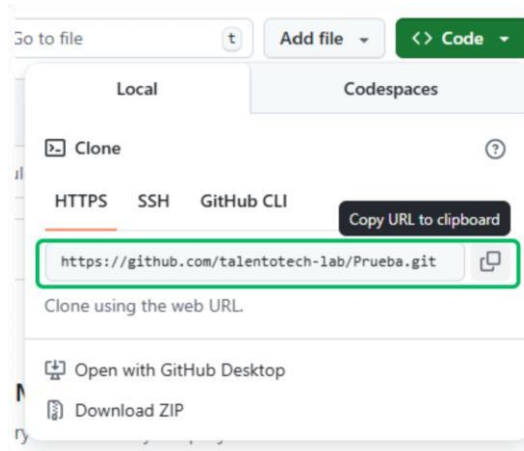
Un **pull request** es una solicitud para fusionar una rama en GitHub. Por lo general se lo utiliza para poder realizar modificación en modo de aporte o propuestas las cuales deberán ser validadas antes de realizar la fusión. Todo esto se realiza desde Github.

Pasos básicos:

1. Ir al repositorio el cual quieres modificar.
2. **Vamos a Fork:** Esto nos permite realizar una copia personal de un repositorio que te permite hacer cambios sin afectar el original.

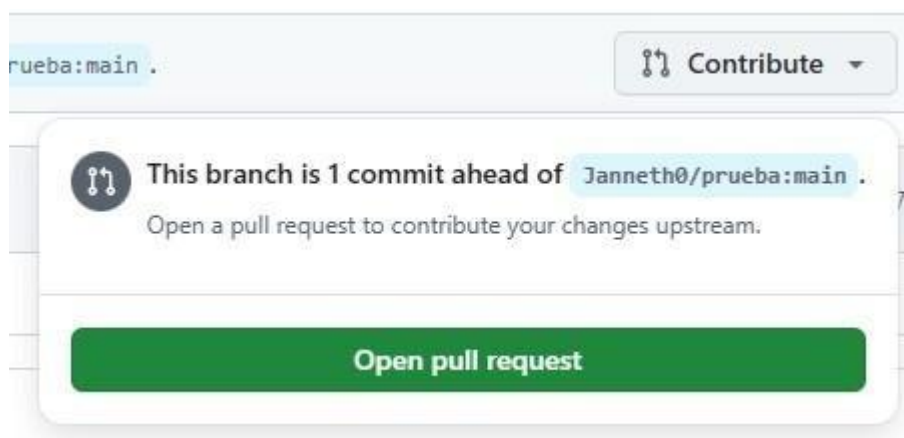


3. Luego realizaremos una clonación del repositorio que queremos modificar escribimos en una nueva terminal y en lo posible una nueva carpeta *git clone* `<url>` la url la obtenemos de Github.
4. Veremos que todo lo que se estuvo trabajando lo descargamos para poder realizar las modificaciones que necesitamos. **Recordá que el código que estás escribiendo no afectará a la rama principal.**

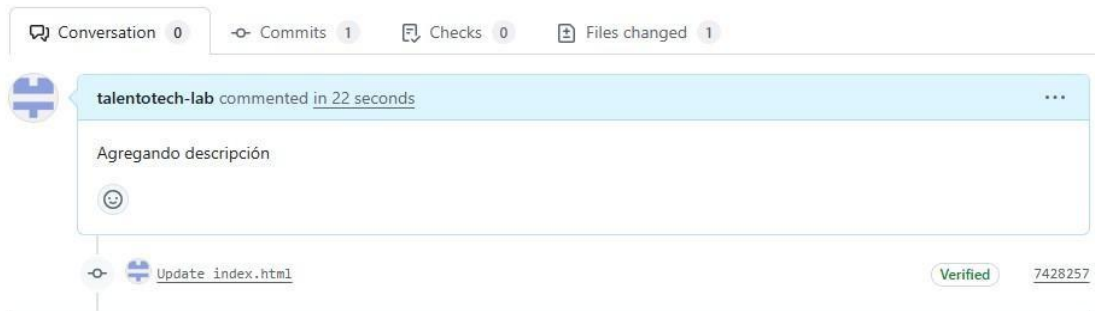


```
PS C:\Users\Talento-tech\Desktop\Proyecto Git> git clone https://github.com/talentotech-lab/Prueba.git
Cloning into 'Prueba'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 9 (delta 2), reused 9 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
Resolving deltas: 100% (2/2), done.
PS C:\Users\Talento-tech\Desktop\Proyecto Git>
```

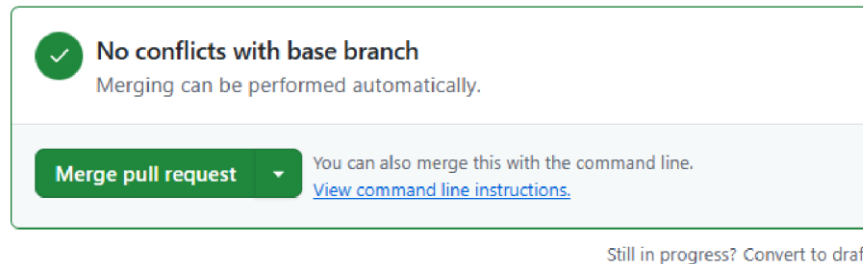
5. Recomendable crear una rama para poder realizar las modificaciones. Una vez realizado los cambios necesarios subir la rama *git push origin nombrerama*
6. Para poder solicitar revisión debemos ir a nuestro proyecto debes seleccionar Contribute y luego Open pull request.



7. Completar título y descripción.



8. Opcional: agregar comentarios o solicitar revisión.
9. Fusionar (merge) desde GitHub.



Diferencia clave:

- **Merge local:** se realiza en tu PC. Con el comando de Git (git merge) fusiona los cambios de una rama a otra en el repositorio local.
- **Pull request:** es una solicitud formal para fusionar código de una rama a otra, un proceso colaborativo que incluye revisión y discusión , se realiza a través de GitHub, ideal para colaboración.

Uso inicial de GitHub Desktop (opcional)

GitHub Desktop permite:

- Clonar repositorios.
- Ver commits y ramas de forma visual.
- Sincronizar cambios con un clic.

Puede resultar útil para quienes recién comienzan y prefieren una interfaz gráfica.

Lo puedes descargar: <https://desktop.github.com/download/>



Cierre conceptual del encuentro

Reconstruimos el flujo completo local → remoto:

1. Trabajo y commits en local.
2. Publicación del código (push).
3. Creación de ramas.
4. Subida de ramas.
5. Inicio y revisión de pull requests.

Este flujo es la base del trabajo colaborativo real y conecta directamente con el Proyecto Integrador.